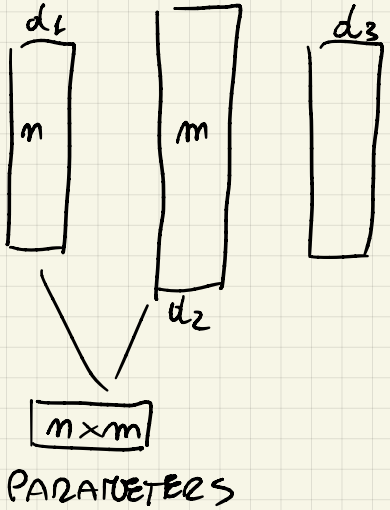
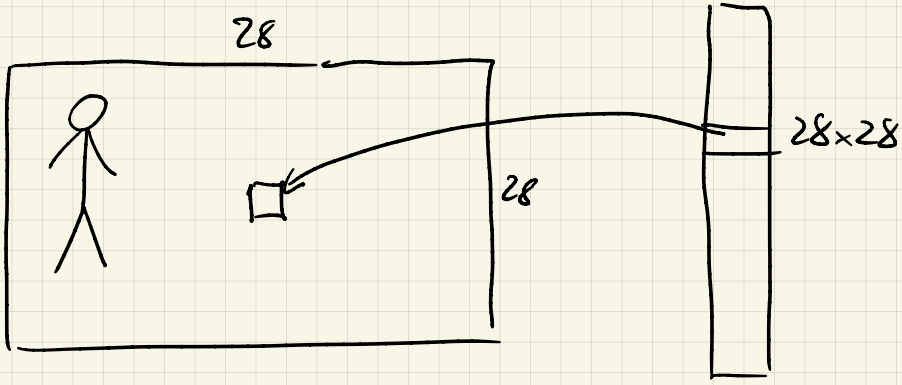
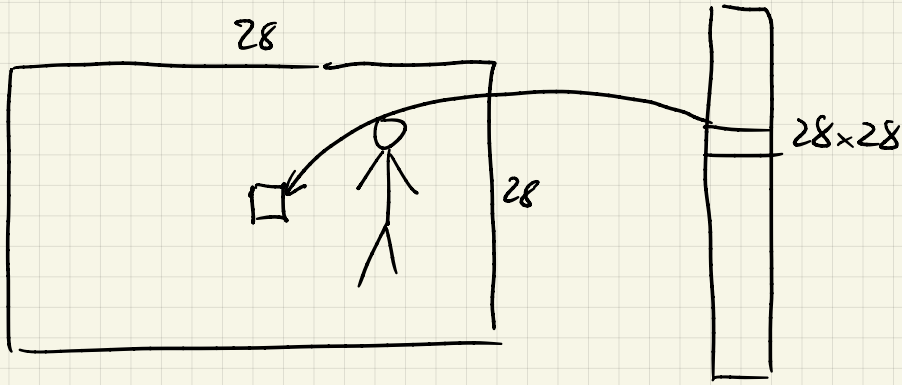
CONVOLUTIONAL NEURAL NETWORKS (CNN)

So far, we have seen feed-forward neural networks, with dence layers, so every unit is linked with every unit of the next layer.



Now, imagine that an image representing a person has to be classified as "person-included" or "dog-included" Let's now take a new image but with the same person transposed on the right of the frame (the NN model has flatten layers)





In both image, the same pixel position is not relevant for the inspection of the image. This makes training and testing very long and complex processes as we have to analyze every single pixel.

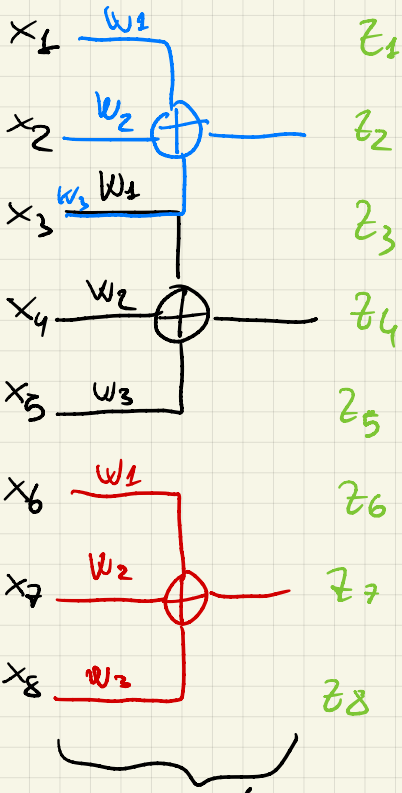
To solve this problem, in CNN it is possible to "move" region by region and analyzing it until our "goal" has been reached (such as identifying a person or not). To study CNNs, we start by a 1D-INPUT example.

N.B.: Not all layers need to be convolutional, but if a layer is, then the network is defined as convolutional

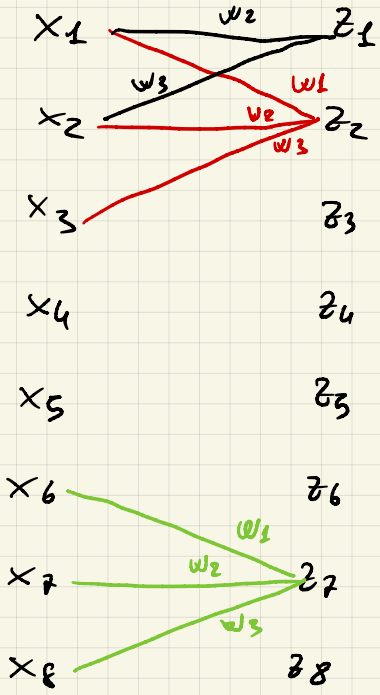
1D-INPUT EXAMPLE

Let's consider 8 inputs. The idea is to construct a "window" that analyzes only some inputs at a time.

N.B.: Biases are considered, but they will be omitted

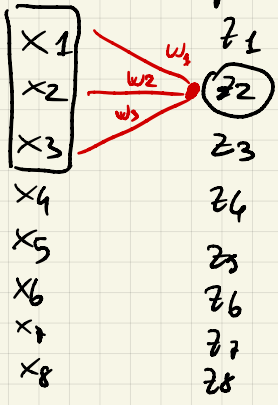


NETWORK STRUCTURE ⟶



What is the difference? The difference is that the result layer are dependent just by some inputs. Also, the weights used are the same anyway.

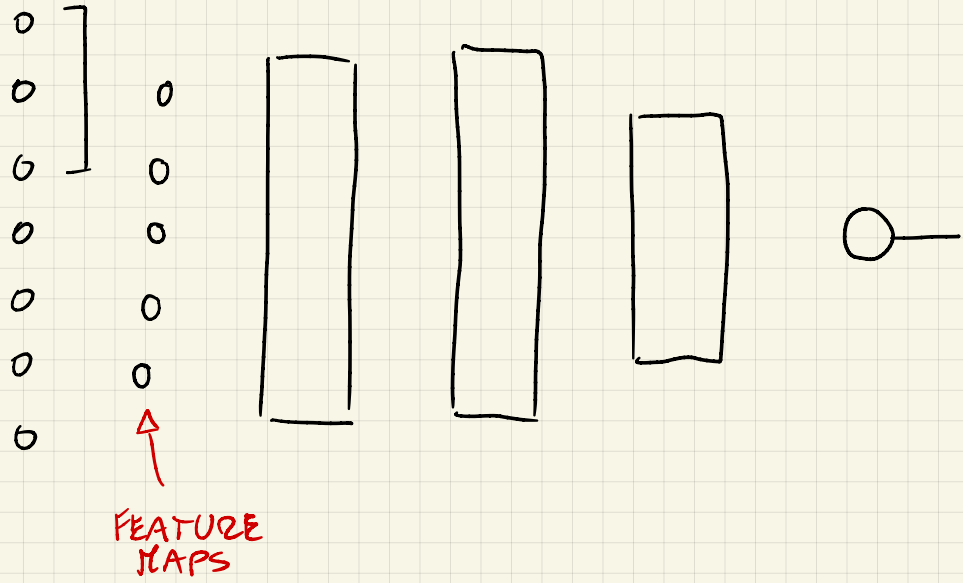
let's see a full computation of a layer.



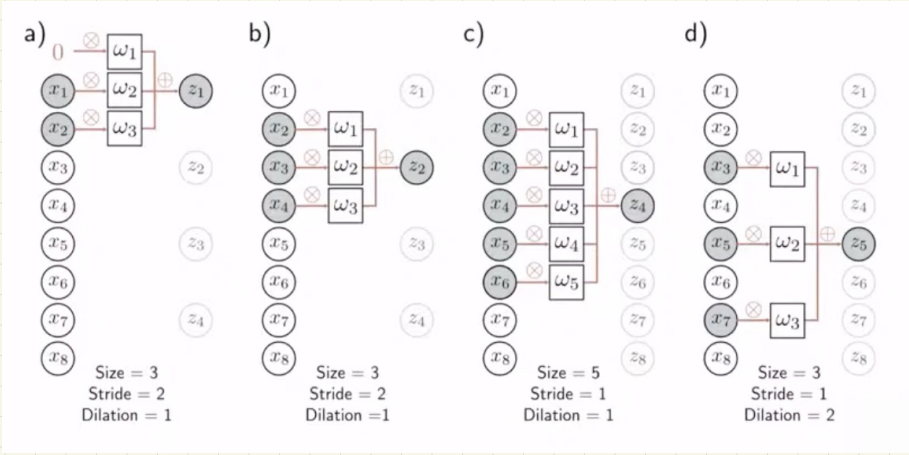
$$z_i = \alpha\left[\beta + w_1 x_1 + w_2 x_2 + w_3 x_3\right]$$

N.B.: This is just a "special case" of a FC neural network. You just assign φ to non connected inputs!

let's see another example. Given an audio fragment, we want to classify it to 1 if there is a honk, 0 otherwise.



What we are doing is just COMPUTING FEATURE MAPS at every convolutional layer. That's because we are "mapping" some 'inputs' properties into single features. However, for every layer, we have to clarify how much are we striding and how many features we consider for every map:

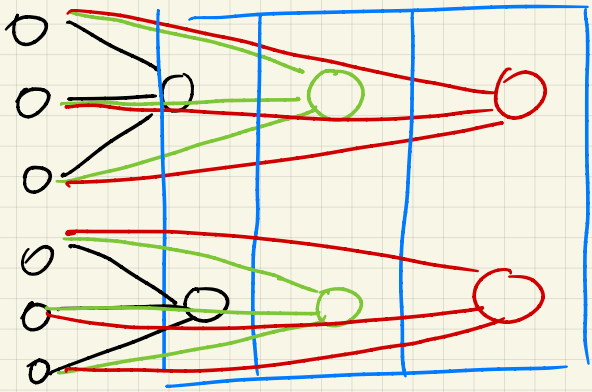


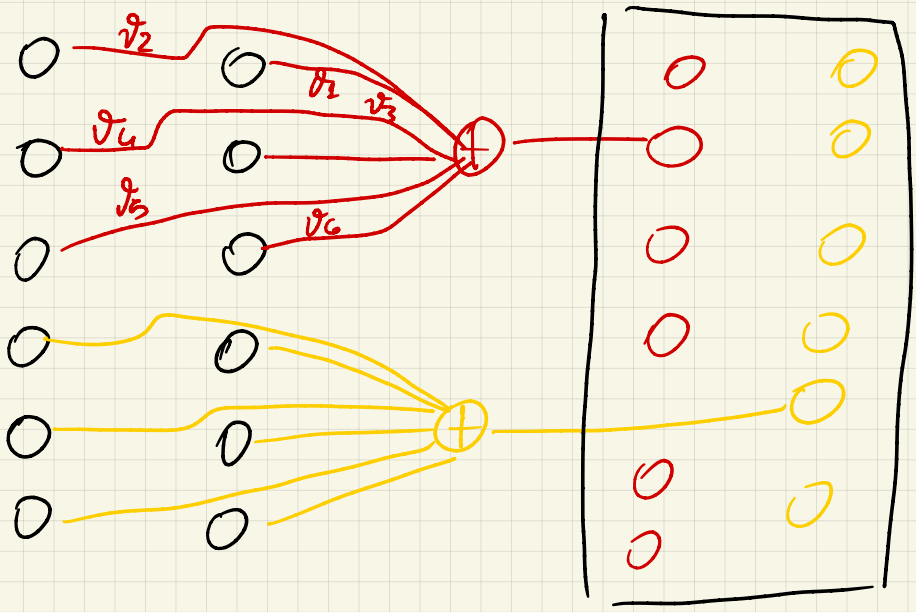
- STRIDE: amount of shift the window does in considering m features
- SIZE: the number of features a window takes (padding included in some cases, as image a))

  ADDING A VALID PADDING MEANS AVOIDING TO PAD WITH ZEROS BY ALWAYS GIVING TO THE WINDOW SOME FEATURES
- DILATION: distance on the layer position between chosen features (dilation = 1 → I choose adjacent features; dilation = 2 → I choose one feature every two)

N.B.: Typically, a filter has odd kernel size. This helps centering the mapping into a specific component.

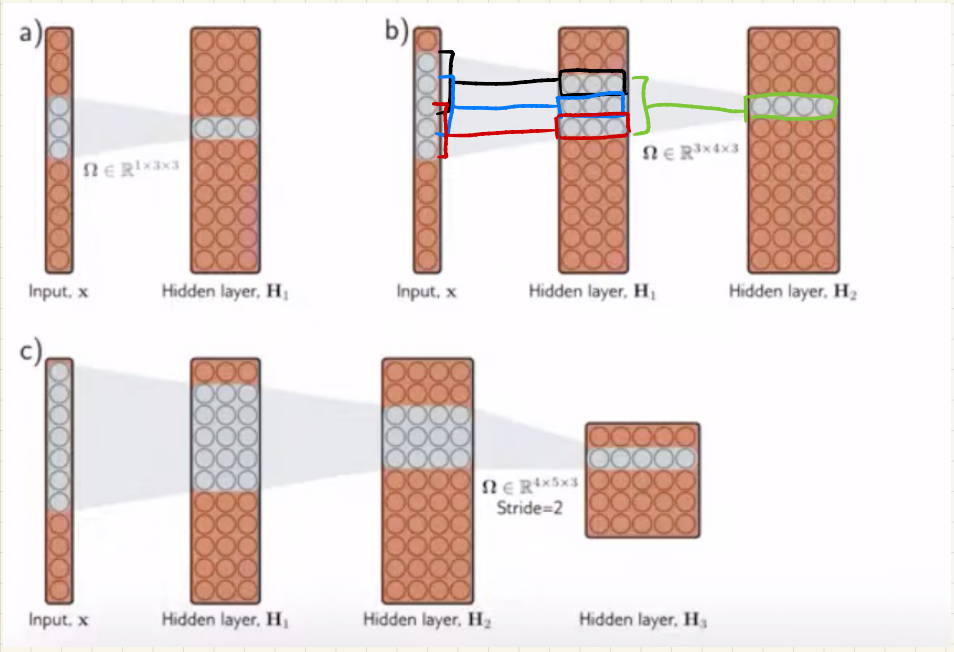
As we have seen, filter weights don't change from a feature to another. To make this process more powerful, it is possible to use DIFFERENT FILTERS with DIFFERENT PARAMETERS. It is also possible to use more filters for the same features by introducing CHANNELS:

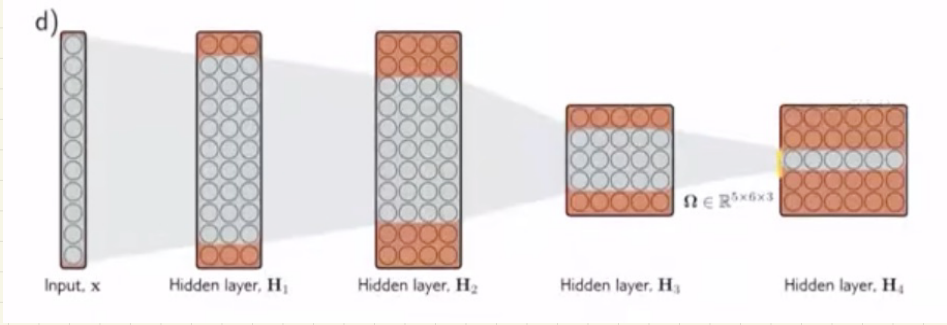




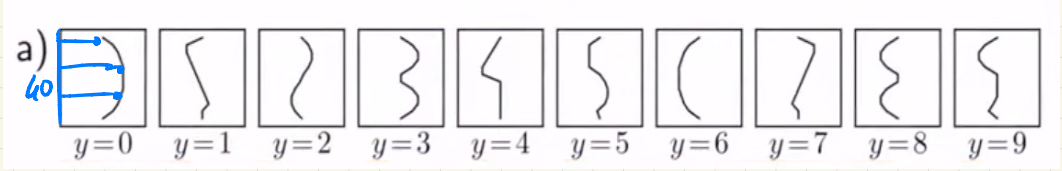
RECEPTIVE FIELDS

This means that I can compute some features into a convolutional layer made by more channels (so a layer could be a TENSOR).





EXAMPLE (MNIST - 1D)

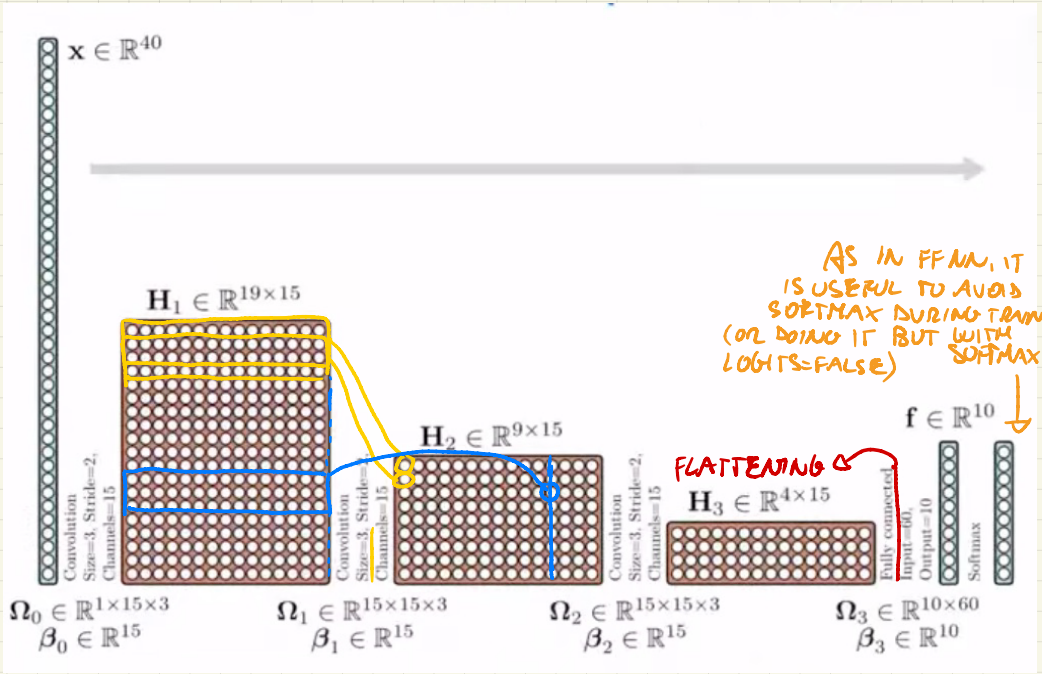


Each image can be represented by a 40 size vector where every element represents the height of the line at the i-th point. We want to classify a vector into 0, 1, 2, 3, ..., 9, based on the image it represents.

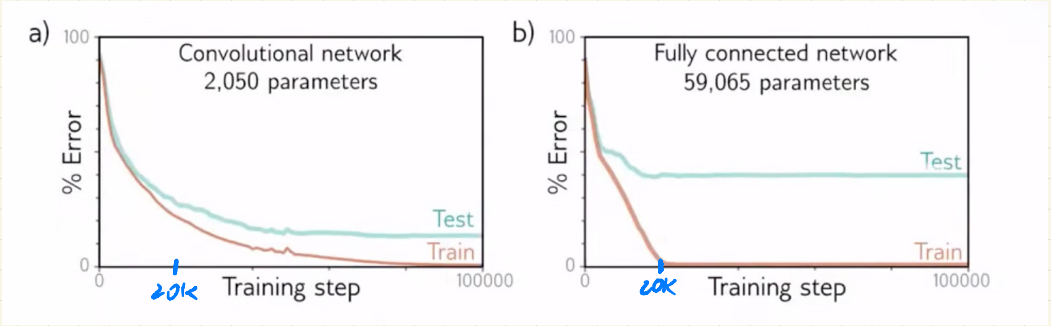
$$\mathbb{R}^{40}\begin{bmatrix}0\\0\\\vdots\\0\end{bmatrix} \qquad \begin{bmatrix}0\\0\\\vdots\\0\end{bmatrix}\mathbb{R}^{10}$$

Of course, it is possible to make it as a feedforward neural network. Here we try to make a cnn.

EXERCISE: MAKE A FEED-FORWARD NN BY YOURSELF!

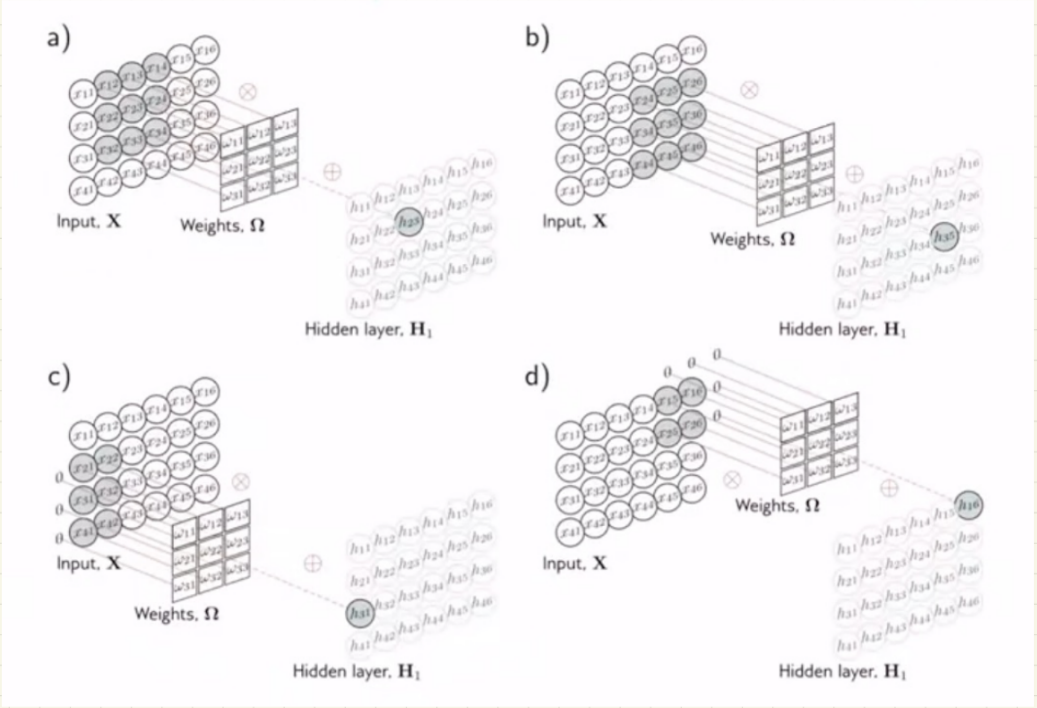


Now, by applying training and testing to this model (with all the specifications needed, like epochs, learning rate,...), we can compare the result with FC NN:



As we see, FC network is much more powerful than CNN. However, if we search on a hypothetical space for a CNN, the power of a FCN doesn't convert into efficiency.

Now, we have just seen a 1-D problem. But the power of CNNs can be seen on 2-D and multi-dimensional inputs problems.



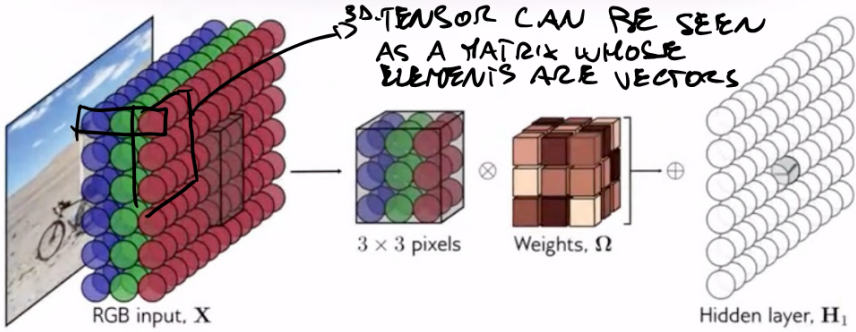
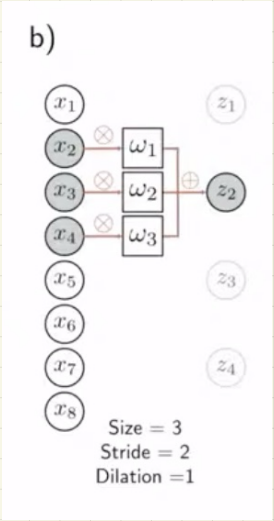


Figure 10.10 2D convolution applied to an image. The image is treated as a 2D input with three channels corresponding to the red, green, and blue components. With a 3×3 kernel, each pre-activation in the first hidden layer is computed by pointwise multiplying the 3×3×3 kernel weights with the 3×3 RGB image patch centered at the same position, summing, and adding the bias. To calculate all the pre-activations in the hidden layer, we "slide" the kernel over the image in both horizontal and vertical directions. The output is a 2D layer of hidden units. To create multiple output channels, we would repeat this process with multiple kernels, resulting in a 3D tensor of hidden units at hidden layer $\mathbf{H}_1$.

In 2D convolution layers, position is relevant (that is why flattening cannot be applied). The process is the same as for 1D-convolution, but with a different architecture.

DOWNSAMPLING

It is the process in which dimension is reduced from a layer to another.



This happens when the kernel is centered by striding by 2 or more elements.
In this case, the kernel is centered by "skipping" x_2, x_4, x_6, x_8 centering.

$z_1 = a(0 + w_2x_1 + w_3x_2 + \beta)$

$z_2 = a(w_1x_2 + w_2x_3 + w_3x_4 + \beta)$

$z_3 = a(w_1x_4 + w_2x_5 + w_3x_6 + \beta)$

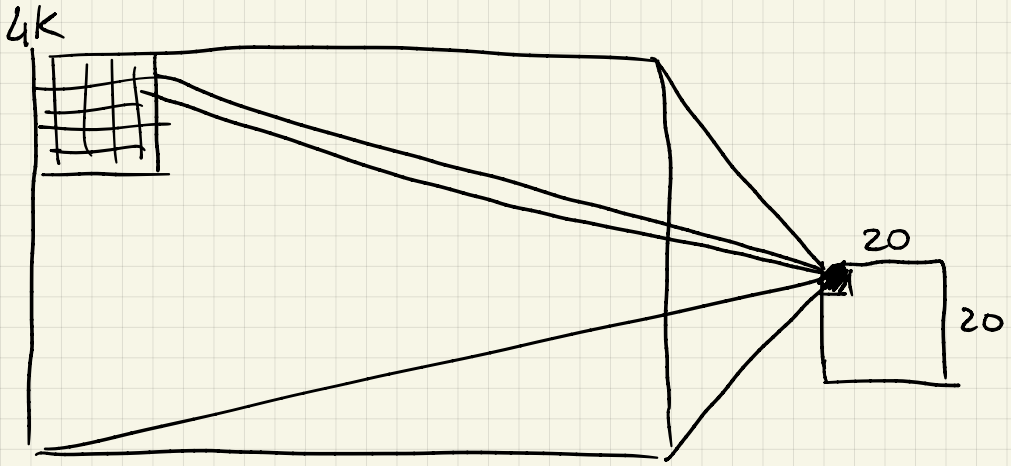
$z_4 = a(w_1x_6 + w_2x_7 + w_3x_8 + \beta)$

There are also other methods for downsampling aside from striding by > 1:

DOWNSAMPLING METHODS:

- stride > 1
- POOLING
  - max POOLING
  - mean POOLING

Let's think about a 4k image. Our goal is to identify a dog in it. We can easily detect it by reducing the 4k image in a 20×20 pixel grid. But how?



The method is to retain all the data in a single pixel by POOLING. We can pool by memorizing the maximum value of the detection for every region (MAX POOLING) or by storing the mean value (MEAN POOLING).

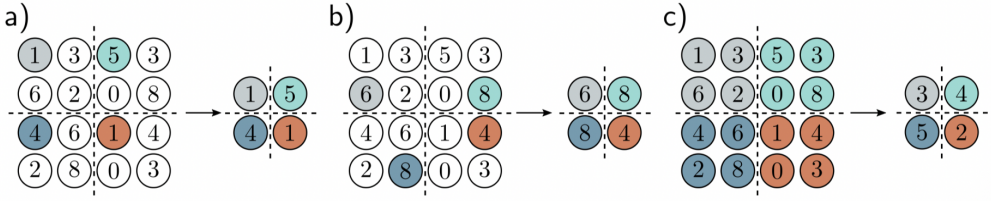
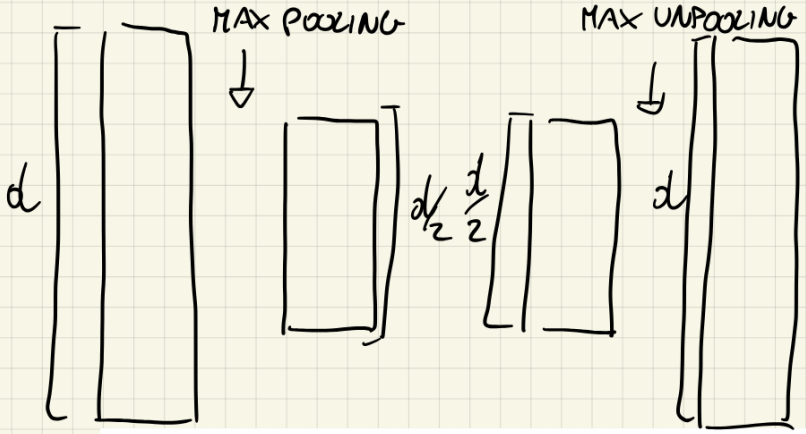


Figure 10.11 Methods for scaling down representation size (downsampling). a) Sub-sampling. The original 4×4 representation (left) is reduced to size 2×2 (right) by retaining every other input. Colors on the left indicate which inputs contribute to the outputs on the right. This is effectively what happens with a kernel of stride two, except that the intermediate values are never computed. b) Max pooling. Each output comprises the maximum value of the corresponding 2×2 block. c) Mean pooling. Each output is the mean of the values in the 2×2 block.

Clearly, it is also to apply the opposite method, which is UPSAMPLING.

UPSAMPLING

It is exactly the opposite of DOWNSAMPLING. To use more values, it is just needed to duplicate the values to fill the desired dimension of the region.



→ It's a type of upsampling applicable when previously some samples were max pooled.

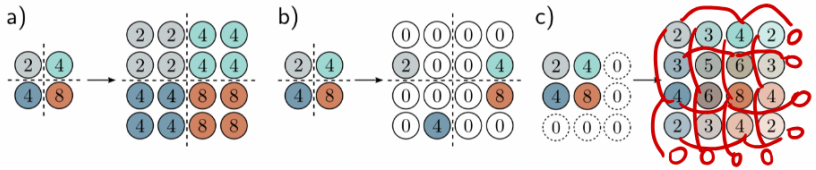


Figure 10.12 Methods for scaling up representation size (upsampling). a) The simplest way to double the size of a 2D layer is to duplicate each input four times. b) In networks where we have previously used a max pooling operation (figure 10.11b), we can redistribute the values to the same positions they originally came from (i.e., where the maxima were). This is known as max unpooling. c) A third option is bilinear interpolation between the input values.

NEEDS TO CARRY THE MAX POOLING INFORMATION FROM WHICH POSITION I PICKED THE VALUE FROM

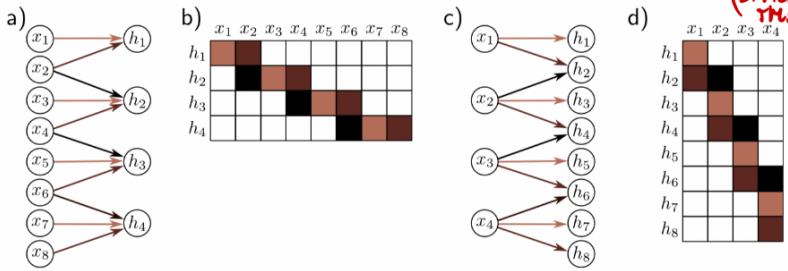


Figure 10.13 Transposed convolution in 1D. a) Downsampling with kernel size three, stride two, and zero-padding. Each output is a weighted sum of three inputs (arrows indicate weights). b) This can be expressed by a weight matrix (same color indicates shared weight). c) In transposed convolution, each input contributes three values to the output layer, which has twice as many outputs as inputs. d) The associated weight matrix is the transpose of that in panel (b).

Also for channels, downsampling and upsampling are applicable in order to reduce or increase the amount of channels in a layer.

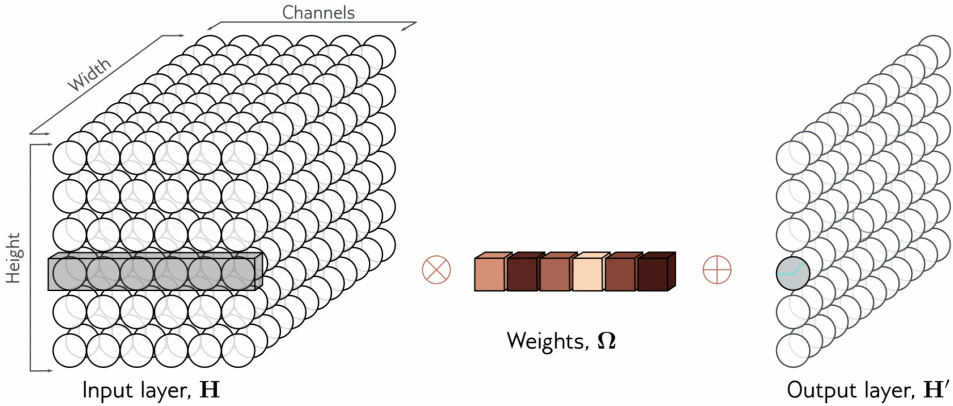
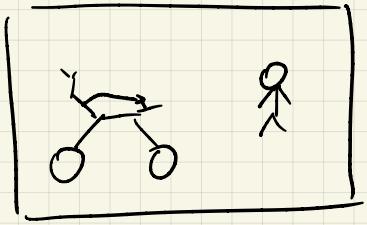


Figure 10.14 1×1 convolution. To change the number of channels without spatial pooling, we apply a 1×1 kernel. Each output channel is computed by taking a weighted sum of all of the channels at the same position, adding a bias, and passing through an activation function. Multiple output channels are created by repeating this operation with different weights and biases.

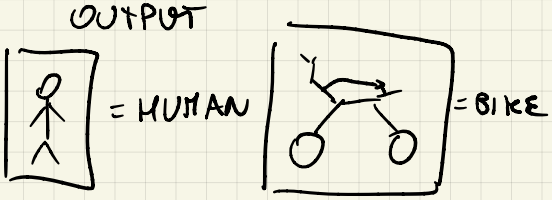
CNN APPLICATIONS

- IMAGE CLASSIFICATION
- OBJECT DETECTION

INPUT IMAGE



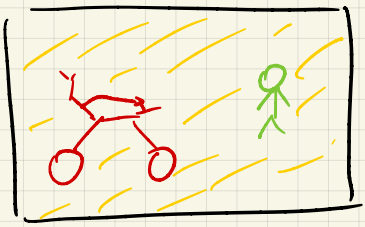
OUTPUT



In object detection, the goal is to detect all objects from an image and classify them by labels.

- SEMANTIC SEGMENTATION

The goal is to classify EVERY SINGLE PIXEL of the input.



CLASSES:
- background
- human
- bike

IMAGE CLASSIFICATION EXAMPLE

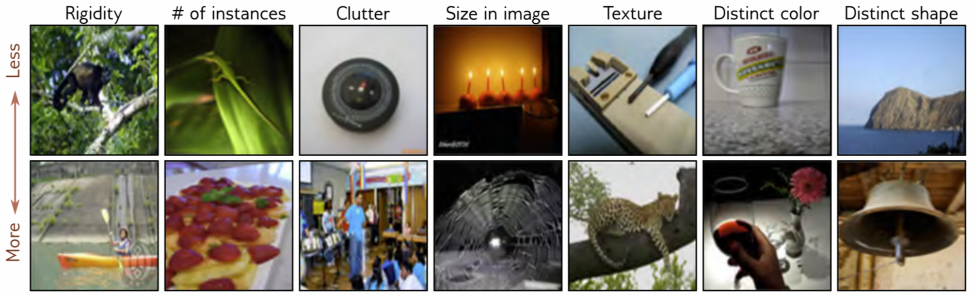
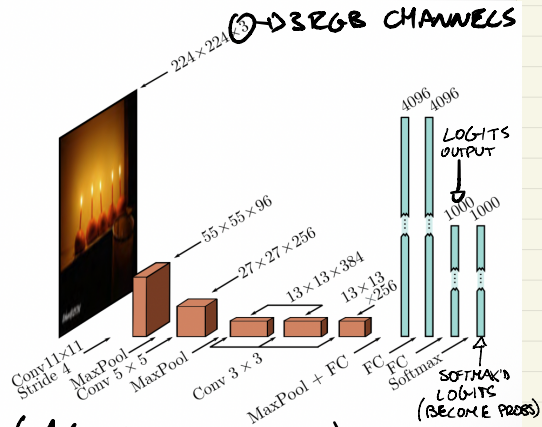


Figure 10.15 Example ImageNet classification images. The model aims to assign an input image to one of 1000 classes. This task is challenging because the images vary widely along different attributes (columns). These include rigidity (monkey < canoe), number of instances in image (lizard < strawberry), clutter (compass < steel drum), size (candle < spiderweb), texture (screwdriver < leopard), distinctiveness of color (mug < red wine), and distinctiveness of shape (headland < bell). Adapted from Russakovsky et al. (2015).

Figure 10.16 AlexNet (Krizhevsky et al., 2012). The network maps a 224×224 color image to a 1000-dimensional vector representing class probabilities. The network first convolves with 11×11 kernels and stride 4 to create 96 channels. It decreases the resolution again using a max pool operation and applies a 5×5 convolutional layer. Another max pooling layer follows, and three 3×3 convolutional layers are applied. After a final max pooling operation, the result is vectorized and passed through three fully connected (FC) layers and finally the softmax layer.



~60M (~16.4% TOP 5 ERROR)

N.B.: The parameters will ALWAYS depend only by the filter dimension.

I.e., a 11×11×96 set of parameters (given by a 11×11 filter with a 96-channel layer) can be always applied independently by the input size.

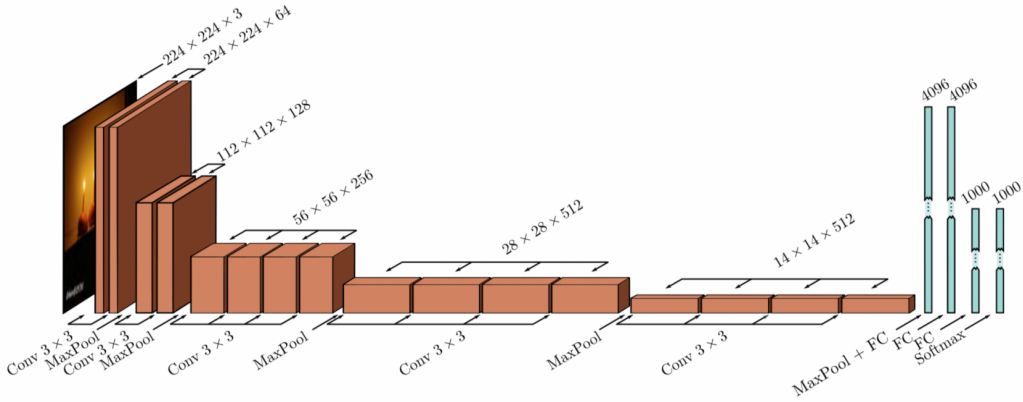


Figure 10.17 VGG network (Simonyan & Zisserman, 2014) depicted at the same scale as AlexNet (see figure 10.16). This network consists of a series of convolutional layers and max pooling operations, in which the spatial scale of the representation gradually decreases, but the number of channels gradually increases. The hidden layer after the last convolutional operation is resized to a 1D vector and three fully connected layers follow. The network outputs 1000 activations corresponding to the class labels that are passed through a softmax function to create class probabilities.

~ 114M (~ 6.8% TOP 5 ERROR)

SEMANTIC SEGMENTATION EXAMPLE

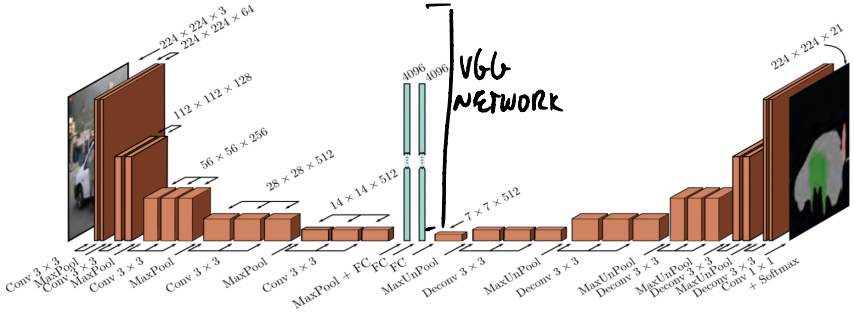


Figure 10.19 Semantic segmentation network of Noh et al. (2015). The input is a 224×224 image, which is passed through a version of the VGG network and eventually transformed into a representation of size 4096 using a fully connected layer. This contains information about the entire image. This is then reformed into a representation of size 7×7 using another fully connected layer, and the image is upsampled and deconvolved (transposed convolutions without upsampling) in a mirror image of the VGG network. The output is a 224×224×21 representation that gives the output probabilities for the 21 classes at each position.

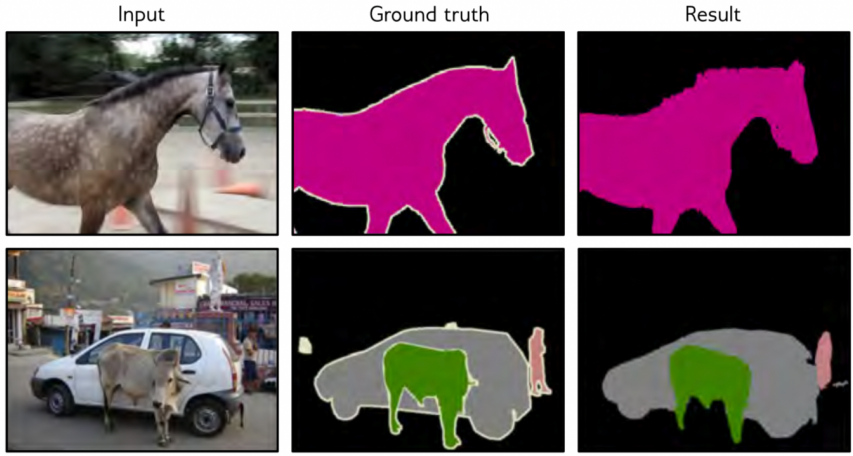


Figure 10.20 Semantic segmentation results. The final result is created from the 21 probability maps by greedily selecting the best class and using a heuristic method to find a sensible binary map based on the probabilities and their spatial proximity. If there is enough evidence, subsequent classes are added, and their segmentation maps are combined. Adapted from Noh et al. (2015).

DIMENSIONALITY REDUCTION

N.B.: This is a general approach, independent from the neural network topic.

Let's imagine a high dimension dataset. It is possible to move it to a much smaller dimension (i.e., resizing images):

$$\mathbb{R}^d \Rightarrow \mathbb{R}^n \quad (n \ll d)$$

There are some available techniques for it. We will see some of them, precisely:

- PCA (Principal Component Analysis); LINEAR APPROACH
- AUTOENCODERS; NON-LINEAR APPROACH

Let's start by defining PCA. The idea is to find a linear transformation in order to reduce the input dimension.

PCA

$S = \{x_1, \dots, x_m\}$, $x \in \mathbb{R}^m$

We bring a matrix $W \in \mathbb{R}^{n \times d}$ s.t.:

$x \longrightarrow Wx$, $Wx \in \mathbb{R}^n$ N.B.: x is the column vector $\vec{x}$

To be more precise:

$$n \left(\overbrace{W}^{d} \right) \quad \begin{matrix} | \\ x \\ | \end{matrix} d \Rightarrow \begin{matrix} | \\ x' \\ | \end{matrix} \Big\} n$$

In general, Wx can be seen as the COMPRESSED VERSION OF x. In opposite, it could be also necessary to UNCOMPRESS the input. It is possible to find another transformation that extracts from Wx the original input:

$y = Wx$ COMPRESSED INPUT

$\hat{x} = Uy = UWx \in \mathbb{R}^d$, $U \in \mathbb{R}^{d,n}$

Actually, this kind of compression has no quality restriction. In our case, we want to guarantee the minimum error by similarity between the input, the compressed input and the uncompressed input.

$$x \longrightarrow Wx \longrightarrow UWx = \tilde{x}$$

As similar as possible

let's see an example:

$$\left.\begin{matrix} \langle 2,4\rangle \\ \langle 3,6\rangle \\ \langle 4,8\rangle \\ \langle 8,16\rangle \\ \langle 20,40\rangle \end{matrix}\right] \quad x \in \mathbb{R}^2 \rightarrow Wx \in \mathbb{R}^1 \rightarrow \tilde{x} = UWx = \begin{matrix} \langle 2\rangle \\ \langle 3\rangle \\ \langle 4\rangle \\ \langle 8\rangle \\ \langle 20\rangle \end{matrix}$$

Of course, x is easily recoverable because there is a strong correlation between the 2 columns of x.
This means that, as soon as some components depend to other, and in a high dimension input it is very possible both linearly or non linearly, the similarity between the inputs, compressed or uncompressed, could be very near considering a proper reduction error.

To minimize the reduction error, it is possible to compute the RECONSTRUCTION ERROR:

$$\sum_{i=1}^{m} |x_i - \overbrace{UWx_i}^{\tilde{x}_i}|^2 : \text{RECONSTRUCTION ERROR}$$

and then minimize it by finding:

$$(1) \quad \underset{U \in \mathbb{R}^{d,m},\, W \in \mathbb{R}^{m,d}}{\operatorname{argmin}} \sum_{i=1}^{m} |x_i - UWx_i|^2$$

W: COMPRESSION MATRIX

U: UNCOMPRESSION MATRIX

It is provable that U and W were strictly related s.t. it

is not necessary to find both.

LEMMA

Let (U, W) be a solution to (1), then:

- the columns of U are orthonormal ($U^TU = I$ of $\mathbb{R}^n$)
- $W = U^T$

then, the problem in can be equivalently written as follows:

$$(2) \quad \underset{U \in \mathbb{R}^{d,n}}{\operatorname{argmin}} \sum_{x_i \in S} |x_i - UU^T x_i|^2$$

THEOREM

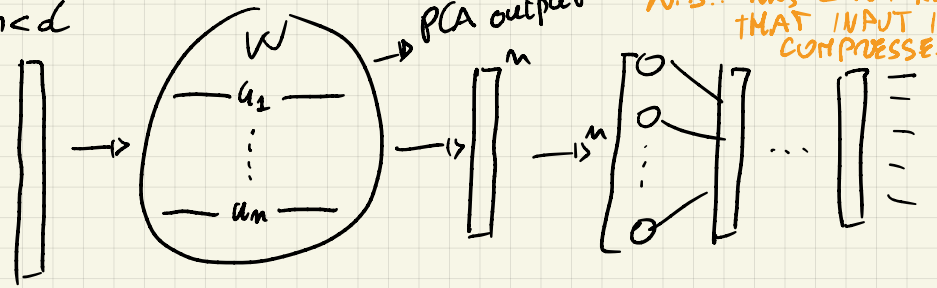
Let $S = \{x_1, \dots, x_m\} \subseteq \mathbb{R}^d$. Let $A = \sum_{i=1}^{m} x_i x_i^T$, and let $u_1, \dots, u_n$ be the eigenvectors of A corresponding to the largest eigenvalues of A. Then, the solution to (2) and (1) by letting $W = U^T$ is the matrix U whose columns are $u_1, \dots, u_n$.

$$U = \begin{bmatrix} | & & | \\ u_1 & \cdots & u_n \\ | & & | \end{bmatrix} \Rightarrow W = U^T$$

$$x \mapsto U^T x \quad \Big] n < d$$

EXAMPLE (BINARY CLASSIFICATION)

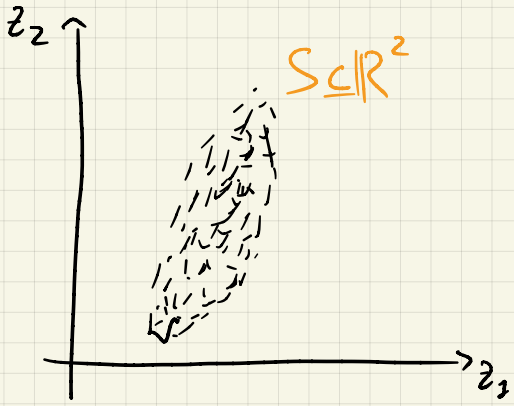
- $n < d$



N.B.: NNs DON'T KNOW THAT INPUT IS COMPRESSED

The purpose of dimensionality reduction is to represent the data in an optimized version, taking note of the reconstruction error.

Also, reconstruction error minimization can be interpreted as variance maximization:

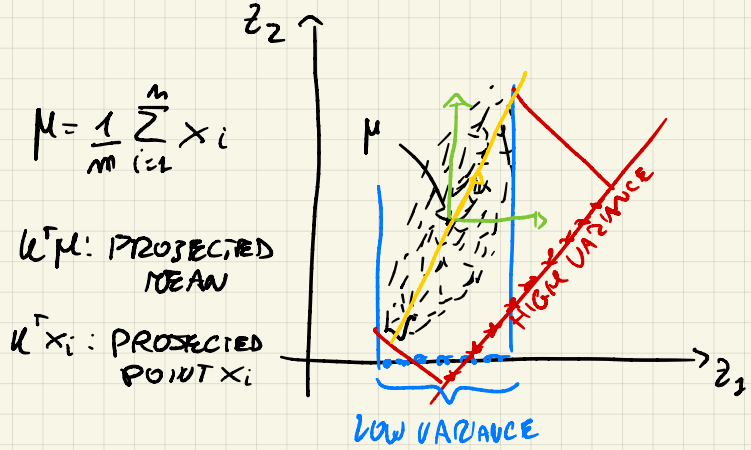


Let's say I want to reduce S dimensionality:

$$S \subseteq \mathbb{R}^2 \to \mathbb{R}^1$$

This projection will have some variance, as some points could collapse on each other onto their projection.

So, by searching for a projection that maximizes variance, I reduce the probability of having collapsing points.



Let's say I am projecting my points onto the yellow line:

VARIANCE OF PROJECTED POINTS

$$\sigma_s^2 = \frac{1}{m}\sum_{i=1}^{m}\left[u^T x_i - u^T\mu\right]^2 = u^T S u$$

$$S = \frac{1}{m}\sum_{i=1}^{m}(x_i-\mu)(x_i-\mu)^T = \frac{1}{m}X^T X, \text{ for } X = \begin{bmatrix}(x_1-\mu)^T \\ \vdots \\ (x_m-\mu)^T\end{bmatrix}$$

By applying normalisation (the matrix X in which every point is sub-tracted by μ), all the points are centered on $\mu = 0$.

Now, I can find the maximum variance:

$$\max_{u \in \mathbb{R}} u^T S u \quad \text{where} \quad u^T u = 1$$

In general ($x_i \in \mathbb{R}^d$, $u \in \mathbb{R}^m$):

$$\max_{u \in \mathbb{R}^m} u^T S u \quad \text{where} \quad u^T u = I \in \mathbb{R}^m$$

$$S = \frac{1}{m} X^T X, \quad X \in \mathbb{R}^d$$

I could also say that:

$$Su = \lambda u$$

s.t. u is an eigenvector of S.

So, if u is eigenvector of S with the highest λ (eigenvalue), then variance is maximal.

EXAMPLE

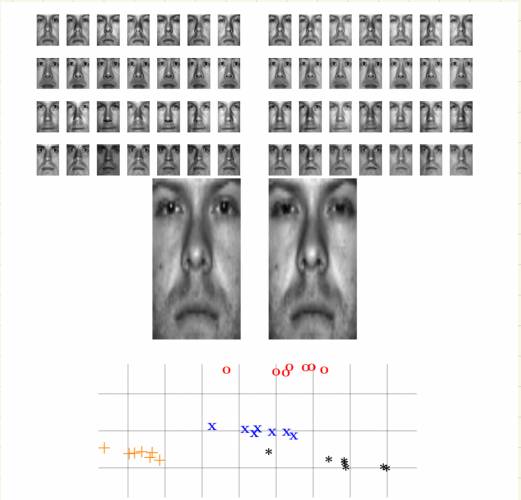
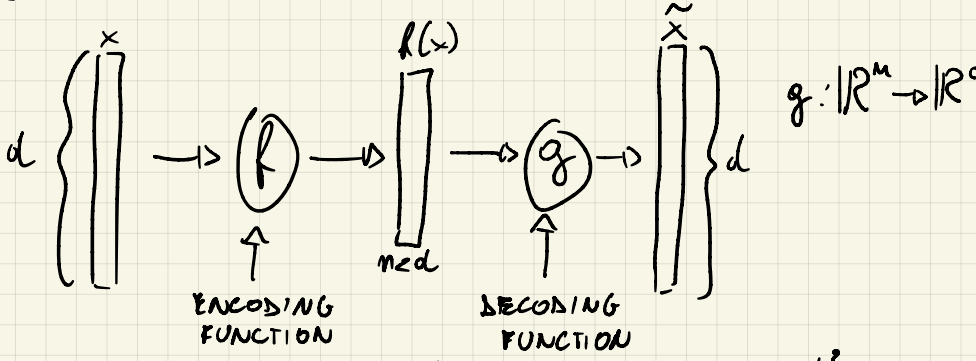


Figure 23.2 Images of faces extracted from the Yale data set. Top-Left: the original images in $\mathbb{R}^{50 \times 50}$. Top-Right: the images after dimensionality reduction to $\mathbb{R}^{10}$ and reconstruction. Middle row: an enlarged version of one of the images before and after PCA. Bottom: The images after dimensionality reduction to $\mathbb{R}^2$. The different marks indicate different individuals.

AUTOENCODERS (NON-LINEAR)

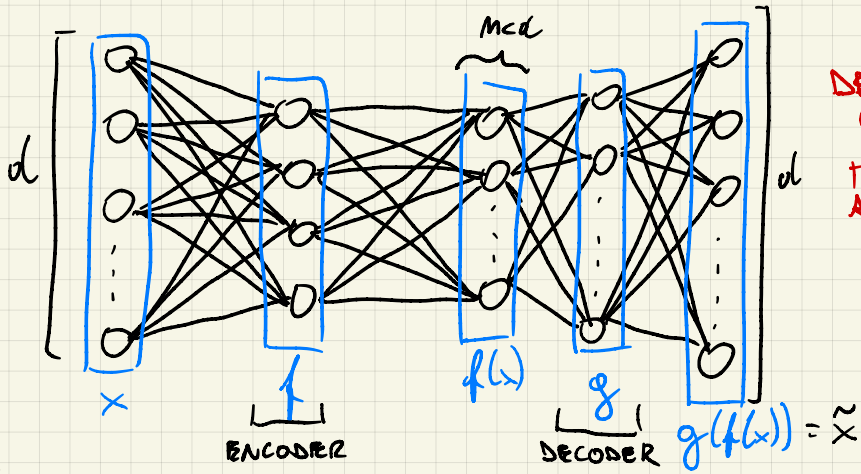
$x \to f(x)$ $\quad x \in \mathbb{R}^d$, $f: \mathbb{R}^d \to \mathbb{R}^m$, $m < d$

In this case, we want to apply reduction of dimensionality, but in a non-linear case:



RECONSTRUCTION ERROR: $L(x) = \sum_{x \in S} |x - g(f(x))|^2$

Now, the problem is to find $f(x)$ and $g(x)$. To solve this, the most useful model is neural networks. So we build a neural network in order to find $f(x)$ and $g(x)$.



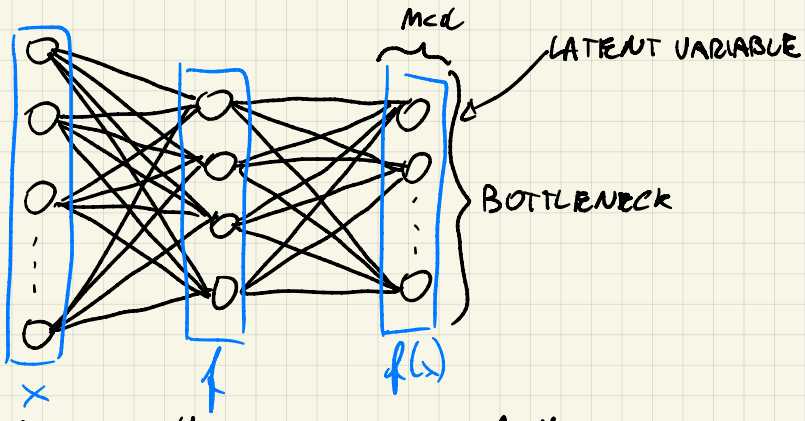
So, we train this NN with the loss function:

$$L(x) = \sum_{i=1}^{m} |x_i - g(f(x_i))|^2$$

RECONSTRUCTION ERROR

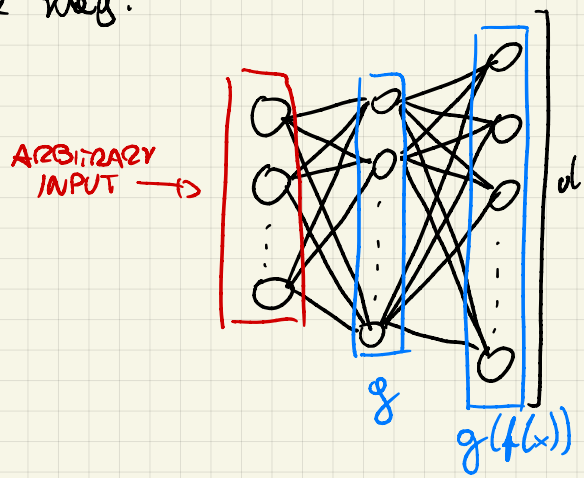
If our goal is to obtain just the reduced version of x, $f(x)$:

- Train the full NN
- Remove g and $g(f(x))$ layers



This is possible because the original NN is fully dependent on how f works (if some data is loss during encoding, g can't recover it). So the cut version of the NN is sufficient enough to compute dimensionality reduction.

It is important to say that $f(x)$ must be a BOTTLENECK, where dimensionality of x is actually reduced. We will often see bottlenecked encoders. But what about the decoder? Can it be used in some way?



Actually, it is possible to give the decoder a random input and generate an output similar to the input. However, it is necessary to continue training the NN in order to make it work. This is a generic example of generative AI.

COVERED IN CH. 14.1 OF
DEEP LEARNING BY GOODFELLOW, BENGIO, COURVILLE

N.B.: if linear functions are applied to f and g, they are basically equal to U and W. So the result will be a convex function where it is possible to find the local minimum or, in some cases, output could be the input itself.

ENSEMBLE METHODS

This topic is about combining some methods previously introduced to implement a more powerful approach to learning.

To start with, let's formalize the idea of WEAK LEARNER

γ-WEAK LEARNABILITY

A learning algorithm A is called a γ-WEAK LEARNER for a class if there exists a function $m_H : (0,1) \to \mathbb{N}$ s.t. for every $\delta \in (0,1)$, for every distribution D over X, and for every labeling function $f: X \to \{-1,+1\}$ if the realizability hypothesis holds (with D, H, f) then when running A on $m \geq m_H(\delta)$ i.i.d. examples generated by D and labelled by f, A returns an hypothesis h s.t. $L_{(D,f)} \leq \frac{1}{2} - \gamma$, with probability $p \geq \delta$